



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический
университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА 5

«Модульное тестирование в Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков
программирования»

Выполнил: студент гр. ИУ5-34Б

(Подпись) (Олейник В. Д.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк Ю. Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Введение

В качестве объектов для тестирования были выбраны функции f1, f2, f3 и класс Unique из лабораторной работы 3-4.

Код, который будет тестироваться:

```
15 def f1(data: list[dict[str, Any]]) -> list[str]:
16
17     return sorted(list(Unique((job['job-name'] for job in data), ignore_case=True)), key=str.lower)
18
19 def f2(data: list[str]) -> list[str]:
20
21     return list(filter(lambda f: str(f).lower().startswith('программист'), data))
22
23 def f3(data: list[str]) -> list[str]:
24
25     return list(map(lambda f: f + ' с опытом Python', data))
```

Рис. 1 – реализация функций f1, f2, f3

```
5 class Unique(object):
8     def __init__(self, items: list[Any] | Generator[Any], **kwargs):
14
15         self.seen: set[Any] = set()
16
17     def __next__(self):
18
19         while True:
20
21             item = next(self.items)
22
23             if self.ignore_case:
24
25                 if isinstance(item, str):
26                     (method) def lower() -> str
27                     Return a copy of the string converted to lowercase.
28                     if item.lower() not in self.seen:
29
30                         self.seen.add(item.lower())
31
32                         return item
33
34             else:
35
36                 if item not in self.seen:
37
38                     self.seen.add(item)
39
40                     return item
```

Рис. 2 – реализация класса для тестирования

В качестве библиотек для тестирования были использованы pytest и behave.

Pytest – для TDD, behave – для BDD

Тестирование с помощью pytest

Тестирование класса. Сценарии:

- удаление дубликатов
- проверка пустого аргумента
- тестирование чувствительности к регистру в двух модификациях

```
7 def test_unique_removes_duplicates():
8     assert list(Unique([1, 2, 1, 3, 2])) == [1, 2, 3]
9
10 def test_unique_empty():
11     assert list(Unique([])) == []
12
13
14 def test_unique_case_sensitive_by_default():
15     assert list(Unique(['a', 'A', 'a'])) == ['a', 'A']
16
17
18 def test_unique_ignore_case():
19     assert list(Unique(['a', 'A', 'b', 'B'], ignore_case=True)) == ['a', 'b']
20
21
22 def test_unique_ignore_case_keeps_numbers():
23     assert list(Unique([1, 'a', 'A', 1], ignore_case=True)) == [1, 'a']
24
```

Рис. 3 – тестирование класса

```

def test_f1_unique_and_sorted():
    data = [{'job-name': 'Python'}, {'job-name': 'python'},
            {'job-name': 'Аналитик'}, {'job-name': 'Java'}]
    assert f1(data) == ['Java', 'Python', 'Аналитик']

def test_f1_empty():
    assert f1([]) == []

def test_f2_filters_by_prefix():
    data = ['Программист', 'программист Python', 'Аналитик', 'Старший программист']
    assert f2(data) == ['Программист', 'программист Python']

def test_f2_no_matches():
    assert f2(['Аналитик']) == []

def test_f3_appends_suffix():
    assert f3(['Программист']) == ['Программист с опытом Python']

def test_f3_empty():
    assert f3([]) == []

```

Рис. 4 – различные сценарии тестирования функций

Тестирование с использованием Mock-объекта

```

64 def test_gen_random_with_mock():
65     with patch('functions.random.randint', side_effect=[7, 8, 9]) as mock_randint:
66         assert list(gen_random(3, 1, 10)) == [7, 8, 9]
67         assert mock_randint.call_count == 3
68         mock_randint.assert_called_with(1, 10)
69

```

Рис. 5 – тестирование с использованием mock

Тестирование с помощью behave

Библиотека behave использует язык Gherkin для описания сценарием тестирования.

```
1 # language: ru
2 Функционал: Уникальные значения
3
4 Сценарий: Удаление дубликатов без учёта регистра
5     Допустим список значений "Python, python, Java"
6     Когда я получаю уникальные значения с игнорированием регистра
7     Тогда результат равен "Python, Java"
8
9 Сценарий: Регистр учитывается по умолчанию
10     Допустим список значений "a, A, a"
11     Когда я получаю уникальные значения
12     Тогда результат равен "a, A"
13
14 Сценарий: Пустой список
15     Допустим пустой список значений
16     Когда я получаю уникальные значения
17     Тогда результат пустой
```

Рис. 6 – сценарии тестирования

```

10 @given('список значений "{text}"')
11 def step_given(context, text):
12     context.items = parse(text)
13
14 @given('пустой список значений')
15 def step_given_empty(context):
16     context.items = []
17
18 @when('я получаю уникальные значения с игнорированием регистра')
19 def step_when_ic(context):
20     context.result = list(Unique(context.items, ignore_case=True))
21
22
23 @when('я получаю уникальные значения')
24 def step_when(context):
25     context.result = list(Unique(context.items))
26
27
28 @then('результат равен "{text}"')
29 def step_then(context, text):
30     assert context.result == parse(text)
31
32
33 @then('результат пустой')
34 def step_then_empty(context):
35     assert context.result == []

```

Рис. 7 – реализация тестирования